HARNESS LAB · MÓDULO 3 · AULA 3.1

CLAUDE.md *enxuto*

Cada linha do arquivo custa atenção em toda tarefa. Inclusive nas que não têm nada a ver com ela. Uma pergunta decide se a linha fica: o agente descobriria isso lendo o código?

`CLAUDE.md` `CLAUDE.old.md` `CLAUDE.local.md` `/context` `/memory` `/doctor`
`/revise-claude-md` `/enxugar-claude-md` `<!-- comentário -->`

VOCÊ SAI COM

Um CLAUDE.md em que toda linha passa no filtro, o original guardado em `CLAUDE.old.md`, a medida de antes e depois no `/context` e a lista do que vai virar hook e skill.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A aula abre com um pedido: se o seu CLAUDE.md não segue as regras desta aula, guarde o que você tem num `CLAUDE.old` e experimente começar de novo. O porquê vem do Módulo 1: instrução demais ao mesmo tempo dilui o peso de cada uma e confunde o modelo. A meta é ficar abaixo de 200 linhas. Quem está com 600 tem trabalho, e vai ganhar qualidade com ele.

68% Com 500 instruções ao mesmo tempo, o melhor modelo testado acerta 68%, com viés para as que aparecem primeiro. IFScale, Distyl AI, arXiv 2507.11538, NeurIPS 2025.

O filtro. O agente descobriria isso lendo o código? Se sim, a linha sai: o que está no `composer.json`, no `package.json` e na estrutura de pastas ele descobre sozinho. Se não, a linha fica: é decisão sua, armadilha do ambiente ou erro que já aconteceu.

O que entra. Comandos exatos (teste com filtro e nome, lint com e sem `--dirty`, e para que serve cada um). Decisão que não é o padrão: regra de negócio em `app/Actions`, que não é o jeito Laravel. Armadilha do ambiente: tudo roda no Docker, comando no host falha. Erro que já aconteceu: datas no banco em UTC, convertidas no front. Fluxo do repositório: branch a partir da `develop`, Conventional Commits. E onde procurar a documentação da API.

O que sai, e é aqui que está o trabalho. Conselho genérico ("código limpo, boas práticas"): o modelo já faz; se a prática é específica, escreva a prática. Aula do framework ("o Laravel usa MVC"): ele escreve Laravel melhor que muito dev. Mapa de pastas: ele roda `ls` e `grep`. Estilo que o linter cobra: é hook. "Nunca rode migrate:fresh": é hook. "IMPORTANTE, sempre rode os testes": se você vive aumentando a ênfase e ele continua errando, a linha é hook. Procedimento de 30 passos: é skill, carregada sob demanda. Coisa de uma tarefa só: vai no pedido.

O antipadrão do arquivo de erros. Histórico de erros dentro do CLAUDE.md, ou num arquivo à parte que o próprio agente alimenta, é premissa de harness antigo, de quando o modelo era muito pior. Contexto histórico compete por atenção. O erro vira uma regra curta: "Datas no banco em UTC; converta só na view."

O esqueleto enxuto

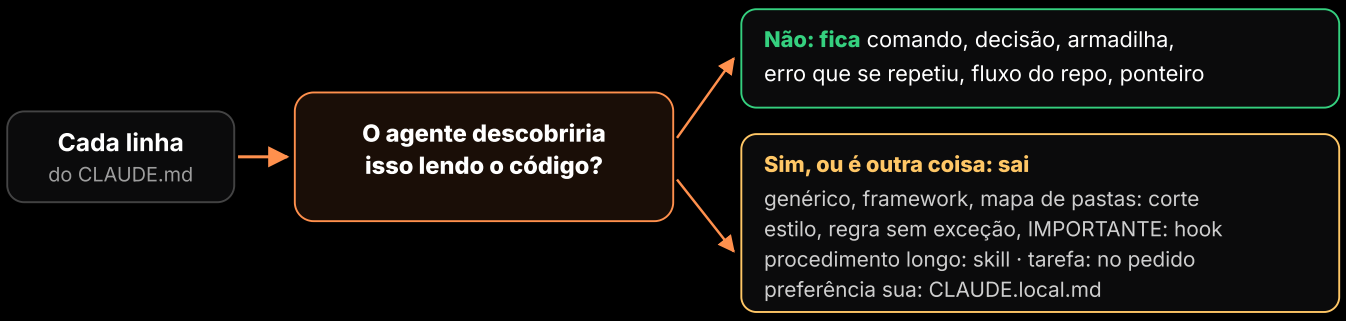
Stack, comandos, decisões de negócio e de padrão, erros que se repetiram (escritos de forma direta) e onde procurar documentação. A stack cabe numa linha de título; o que sai é a lista de versões, que está no `composer.json`. Nunca automático: o erro só entra quando você percebeu que ele se repete.

Como saber se funciona

Mande `oi` e rode `/context`. `/memory` para ver CLAUDE.md espalhado em subpasta. Uma pergunta-teste. Numa sessão longa, pergunte sobre a regra. E veja se as correções da 1.3 voltam.

> *"Claude Code loads CLAUDE.md into every session, which means every line in it takes a little of Claude's attention on every task, whether it's relevant or not. That's why the file works best when it only holds what Claude couldn't work out from the code itself."*
>
> Documentação oficial do Claude Code · Best practices · Keep CLAUDE.md short

2 O que a aula *não teve tempo de dizer*



ALÉM DA AULA · O NÚMERO DO /CONTEXT QUE IMPORTA

O total que o `/context` mostra depois de um `oi` soma system prompt, ferramentas, servidores MCP, skills e agentes. Quem mede o CLAUDE.md é a linha **Memory files**, e logo abaixo vem a tabela com cada arquivo de memória carregado e quantos tokens ele custa. Total alto com Memory files pequeno quer dizer que o peso está em outro lugar (MCP e skills são da aula 3.5). Teste feito para este material: `claude -p "/context"` imprime a mesma tabela no terminal, sem abrir sessão (precisa estar logado). Bom para anotar o antes e o depois.

Arquivo acima da meta também avisa sozinho: a doc diz que você vê um aviso no início da sessão e no `/status` quando um arquivo de instrução passa do tamanho recomendado.

Fontes: Explore the context window ("run /context for a live breakdown by category"); Memory, "My CLAUDE.md is too large".

ALÉM DA AULA · CLAUDE.OLD: COPIE, NÃO MOVA

O Claude Code carrega `CLAUDE.md`, `.claude/CLAUDE.md` e `CLAUDE.local.md`. Um `CLAUDE.old.md` não entra no contexto, então o backup é de graça. Mas cuidado com `mv`: se o repositório tem um `AGENTS.md` (de Cursor, Codex ou do Laravel Boost), sem CLAUDE.md o Claude Code passa a ler o `AGENTS.md` no lugar. Teste feito para este material: renomeado o CLAUDE.md, o `/context` passou a listar o `AGENTS.md` em Memory files. Por isso o passo a passo usa `cp` e você corta no próprio arquivo.

Fonte: Memory, "AGENTS.md": com CLAUDE.md presente, o AGENTS.md não é lido; sem ele, é (Claude Code 2.1.277+).

ALÉM DA AULA · NOTA PARA HUMANO NÃO GASTA CONTEXTO

"Este bloco foi pedido pelo time de pagamentos" é útil para quem mantém o arquivo e inútil para o modelo. Ponha num comentário HTML de bloco: `<!-- ... -->`. O Claude Code tira esses comentários antes de mandar o CLAUDE.md para o modelo. Teste feito para este material: o `CLAUDE.md.exemplo` da aula, com 29 linhas no arquivo e 10 delas de comentário, custou os mesmos 361 tokens no `/context` com e sem os comentários. Dentro de bloco de código (```) o comentário fica.

Fonte: Memory, "Block-level HTML comments (<!-- maintainer notes -->) in CLAUDE.md files are stripped before the content is injected into Claude's context."

ALÉM DA AULA · POR QUE IMPORTANTE ATRAPALHA, E NÃO SÓ "NÃO AJUDA"

A doc lista a ênfase em maiúscula como conselho que ficou velho: "Current ones follow a plainly worded line, and piling on emphasis mostly makes Claude over-apply the rule." Ou seja, o "NUNCA" em caixa alta faz o modelo aplicar a regra onde ela não cabe. Escreva a linha em tom normal. Se mesmo assim ele erra, é hook ou `permissions.deny`. E o critério da doc para o erro entrar no arquivo é o mesmo da aula: "Anything you've had to correct twice already."

Fonte: Best practices, "What you can stop doing" e "Keep CLAUDE.md short".

ALÉM DA AULA · CLAUDE.MD DE SUBPASTA: QUANDO FAZ SENTIDO

A aula recomenda deixar tudo na raiz, e é o certo para a maioria dos projetos. O caso que muda: monorepo, ou uma pasta com regras que só valem ali. O CLAUDE.md de subpasta não entra no começo da sessão; carrega quando o Claude lê, escreve ou edita um arquivo daquela pasta. Duas pegadinhas: depois de uma compactação ele só volta quando a pasta é tocada de novo, e um arquivo de subpasta grande demais chega cortado (40.000 caracteres com janela de 200K). Para regra que vale para um tipo de arquivo, a ferramenta certa é `.claude/rules/` com `paths:`, que é da aula 3.2.

Fontes: Memory, "How CLAUDE.md files load" e "Claude ignores the end of my subdirectory CLAUDE.md"; Explore the context window, "What survives compaction".

ALÉM DA AULA · AS FERRAMENTAS QUE ESCREVEM POR VOCÊ TAMBÉM PASSAM NO FILTRO

O `/revise-claude-md` (plugin oficial `claude-md-management`) relê a sessão e propõe linhas novas, em diff, e só grava o que você aprovar. Combina com a aula porque não é automático. Mas a lista do que ele procura inclui "Code style patterns followed", que a aula manda para o lint. E, no comando 1.0.0, o arquivo pessoal aparece como `.claude.local.md`, nome que o Claude Code não carrega: o certo é `CLAUDE.local.md`. O `/doctor` propõe cortes do que dá para tirar do código, e o `/doctor prompt-audit` (2.1.283+) aponta instrução escrita para modelo antigo, arquivo que não existe e contradição entre arquivos. Os três sugerem; quem decide é a pergunta da aula.

Fontes: `commands/revise-claude-md.md` do plugin, em github.com/anthropics/claude-plugins-official; Memory, "Audit your instruction files".

3 Faça *no seu repositório*

A aula não instala hook nem settings. O que entra no seu projeto é um arquivo de exemplo (`CLAUDE.md.exemplo`, que não substitui o seu) e dois plugins: o `enxuto`, do curso, com a skill que aplica o filtro, e o `claude-md-management`, oficial da Anthropic. Nada é gravado no seu CLAUDE.md sem você aprovar.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Meça e guarde o original antes de cortar.

Anote o número de linhas para comparar depois (e para mandar nos comentários da aula). O backup é uma cópia: o seu CLAUDE.md continua no lugar. O `-n` não sobrescreve um `CLAUDE.old.md` que já exista, então refazer o passo depois de cortar não apaga o original.

Terminal · na raiz do seu projeto — shell

```
wc -l CLAUDE.md
cp -n CLAUDE.md CLAUDE.old.md
```

O seu está em `.claude/CLAUDE.md`? Troque o caminho nos dois comandos e no passo 9. Ainda não tem CLAUDE.md? Pule o backup e comece do `CLAUDE.md.exemplo` do passo 3.

Numa sessão nova, a medida de antes. Anote a linha **Memory files** e confira, no `/memory`, se há CLAUDE.md em subpasta ou no `~/.claude/` que você esqueceu:

Dentro do Claude Code · sessão nova — sessão

```
> oi
/context
/memory
```

3 Instale o esqueleto enxuto da aula.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 3.1
```

Cria só o `CLAUDE.md.exemplo` na raiz. É para ler e copiar os blocos, não para usar como está: troque o conteúdo pelo do seu projeto e apague o arquivo quando terminar.

CLAUDE.md.exemplo — instalado pelo instalar.sh · não copie do PDF

```
<!-- nota para quem mantém: não vai para o contexto -->
# Loja · Laravel 12 + Livewire + Pest

## Comandos
- Tudo via Sail: `./vendor/bin/sail artisan ...`, nunca `php artisan` direto.
- Um teste: `./vendor/bin/sail artisan test --filter=Nome`
- Lint do que mudou: `./vendor/bin/sail bin pint --dirty`

## Decisões
- Regra de negócio em app/Actions, uma classe por caso de uso.
- Dinheiro em centavos (int). Float não entra em valor de pedido.

## Erros que já aconteceram
- Datas no banco em UTC; converta só na view com `->timezone(config('app.tz'))`.
  (Converter no model dobrou o fuso no relatório de vendas.)
- Fila de e-mail roda em redis, não sync; teste com `Mail::fake()`.
  (Teste sem fake mandou e-mail real.)

## Onde procurar
- Contrato da API: docs/api.md. Leia antes de mexer em rota.
```

Repare no último bloco: o ponteiro é texto, sem `@`. Escrito `@docs/api.md`, o arquivo inteiro seria importado e carregaria junto em toda sessão. Para citar um caminho com `@` sem importar, ponha entre crases.

4 Instale o plugin do curso.

Terminal · qualquer pasta — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin marketplace update harness-lab
claude plugin install enxuto@harness-lab
```

5 Instale o plugin oficial de manutenção do CLAUDE.md.

O marketplace oficial é registrado sozinho na primeira sessão interativa. Numa máquina onde isso ainda não aconteceu, o `install` falha sem o `add`. Rodar o `add` de novo não faz mal.

Terminal · qualquer pasta — claude

```
claude plugin marketplace add anthropics/claude-plugins-official
claude plugin install claude-md-management@claude-plugins-official
```

Prefere fazer de dentro do Claude Code? O mesmo, pelo `/plugin`:

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin marketplace update harness-lab
/plugin install enxuto@harness-lab
/plugin install claude-md-management@claude-plugins-official
/reload-plugins
```

6 Passe o filtro com a skill.

Numa sessão interativa: a skill pergunta antes de gravar, e no `claude -p` não há quem responda (no modo padrão, toda edição pede permissão). Se a sessão já estava aberta, recarregue antes:

Dentro do Claude Code · na raiz do projeto — sessão

```
/reload-plugins
/enxugar-claude-md
```

A skill roda o `filtro.py` (só lê arquivos), confere no código cada linha que parece descobrível, e devolve: a tabela linha por linha com o destino (fica, sai, reescrever, vira hook, vira skill, vai para o pedido, `CLAUDE.local.md`), o arquivo novo nos quatro blocos, o antes e depois em linhas e as listas "anote para virar hook" e "anote para virar skill". Ela não cria hook nem skill. Só grava depois do seu sim, e guarda o original em `CLAUDE.old.md` se você ainda não guardou (se o backup do passo 2 já está lá, igual ao original, ela não faz outro). Teste feito para este material: no CLAUDE.md inchado do `teste/projeto/` do repositório, a proposta foi de 69 para 18 linhas.

7 Meça de novo e faça a pergunta-teste.

Numa sessão nova: `oi`, `/context` e compare a linha Memory files. Depois, um pedido pequeno que deveria acionar uma regra que ficou (este aciona a das datas do exemplo; troque pelo seu caso):

Dentro do Claude Code · sessão nova — sessão

```
> oi
/context
> Mostre a data de entrega do pedido na tela de detalhes
/doctor prompt-audit
```

Ele converteu na view sem você lembrar? Repita a pergunta depois de uma hora de sessão: a obediência muda ao longo da conversa. O `/doctor prompt-audit` só relata; nada muda até você pedir.

8 No fim de uma sessão que ensinou algo.

Rode `/revise-claude-md` (do `claude-md-management`). Ele mostra o diff proposto e pergunta antes de gravar. Passe cada linha no filtro, recuse estilo de código (é lint) e, se ele sugerir `.claude.local.md`, peça `CLAUDE.local.md`.

9 Confira que ficou tudo no lugar.

Terminal · na raiz do seu projeto — claude

```
claude plugin list
wc -l CLAUDE.md CLAUDE.old.md
```

A lista precisa ter `enxuto@harness-lab` e `claude-md-management@claude-plugins-official`, os dois `enabled`. O `wc` mostra o antes e o depois lado a lado. Depois, commit do CLAUDE.md: ele é do time.

4 Arquivos que *reforçam a aula*

`CLAUDE.md.exemplo`

O esqueleto do slide, pronto: quatro blocos, 19 linhas no contexto, erro escrito como regra + jeito + porquê e notas para humanos em comentário HTML. Passa no próprio filtro com zero marcas.

No repositório: modulo-3-regra/aula-3.1-claude-md-enxuto/projeto/CLAUDE.md.exemplo

`checklist-de-corte.md`

O filtro em forma de lista para passar linha por linha: o que fica, o que sai e para onde vai (corte, hook, skill, pedido, `CLAUDE.local.md`, rule com `paths:`, comentário), o antes e depois de um erro reescrito e a medida de antes e depois.

No repositório: modulo-3-regra/aula-3.1-claude-md-enxuto/modelos/checklist-de-corte.md

`esqueleto-por-stack.md`

O mesmo esqueleto para Node/Next.js, Python/Django e Ruby/Rails. Mostra que o que muda entre stacks é o conteúdo, não os blocos.

No repositório: modulo-3-regra/aula-3.1-claude-md-enxuto/modelos/esqueleto-por-stack.md

`skills/enxugar-claude-md/`

A skill e o `filtro.py`. O script lista tudo o que carrega (raiz, `CLAUDE.local.md`, subpastas, rules, `@imports`), conta só o que vai para o contexto e marca ênfase, regra sem exceção, história de erro, conselho genérico, estilo, aula de framework, mapa de pastas e procedimento longo. Avisa também `AGENTS.md` ignorado, `CLAUDE.local.md` fora do `.gitignore` e import quebrado.

No repositório: plugins/enxuto/skills/enxugar-claude-md/

`teste/projeto/CLAUDE.md`

Um CLAUDE.md inchado de propósito, com todos os falsos padrões da aula, num projeto Laravel falso. Treine o olho nele antes de cortar o seu, e compare com a proposta da skill.

No repositório: modulo-3-regra/aula-3.1-claude-md-enxuto/teste/projeto/CLAUDE.md

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
Renomeou o CLAUDE.md e o agente passou a seguir regras de outro arquivo	Sem CLAUDE.md, o Claude Code lê o `AGENTS.md` da raiz	Use `cp` para o backup e corte no próprio CLAUDE.md
`Plugin "claude-md-management" not found in marketplace`, sugerindo `marketplace update`	Na máquina, o marketplace oficial ainda não foi registrado; o update não resolve	`claude plugin marketplace add anthropics/claude-plugins-official` e instale de novo
`/enxugar-claude-md` não aparece no menu	Plugin instalado com a sessão aberta	`/reload-plugins` ou sessão nova

SINTOMA	CAUSA	CORREÇÃO
Cortou para 30 linhas e o `/context` quase não mudou	O total soma MCP, skills e ferramentas	Compare a linha Memory files, não o total
O ponteiro "Leia @docs/api.md" fez o contexto crescer	`@` fora de crase importa o arquivo inteiro	Ponteiro em texto, ou entre crases
Linha pessoal do `/revise-claude-md` nunca é seguida	Ele gravou em `.claude.local.md`, que não carrega	Mova para `CLAUDE.local.md` e ponha no `.gitignore`
A skill propôs e nada foi gravado no `claude -p`	No `claude -p` ninguém aprova a edição nem responde o "pode gravar?" da skill	Rode a skill numa sessão interativa

6 Você sai *com*

- [ ] `CLAUDE.old.md` com o original, e o número de linhas de antes e de depois (`wc -l`).
- [ ] A linha Memory files do `/context` anotada antes e depois de cortar.
- [ ] Um CLAUDE.md nos quatro blocos, com cada linha passando no filtro e abaixo de 200 linhas.
- [ ] Cada erro cometido escrito como regra + jeito + porquê, numa linha, sem história.
- [ ] A lista "anote para virar hook" e a lista "anote para virar skill" (aula 3.3).
- [ ] Uma pergunta-teste que aciona uma regra, respondida certo numa sessão nova.
- [ ] `claude plugin list` com `enxuto@harness-lab` e `claude-md-management@claude-plugins-official`.

PROMETIDO NA AULA

- ✓ "Crie um CLAUDE.old e jogue o que você já tem lá": passo 2 da seção 3 (`cp -n CLAUDE.md CLAUDE.old.md`), e a skill faz o mesmo antes de gravar.
- ✓ "Manda um oi, dá um /context, vê como ficou": passos 2 e 7 da seção 3, com a linha certa para olhar no gap "O número do /context que importa".
- ✓ "Manda nos comentários quantas linhas tinha e quantas pretende deixar": o `wc -l` dos passos 2 e 9.
- ✓ "Vou te dar uma skill para construir o seu CLAUDE.md": é da aula 3.2. Aqui você já tem a que corta, `/enxugar-claude-md`.

7 Glossário

CLAUDE.md	Arquivo de instruções que o Claude Code carrega no início da sessão (o de subpasta, quando ele mexe naquela pasta). O da raiz é do time e vai para o git.
CLAUDE.local.md	Mesmo papel, pessoal: carrega junto, vai no `.gitignore`.
CLAUDE.old.md	Nome sem significado para o Claude Code. Por isso serve de backup sem gastar contexto.
Memory files	Linha do `/context` com o custo, em tokens, dos arquivos de instrução carregados.
@import	`@caminho` no CLAUDE.md: carrega o arquivo junto. Organiza, não economiza.
Falso padrão	Linha que parece útil e só ocupa espaço (genérico, framework, mapa) ou está no lugar errado (é hook, skill ou pedido).
Pergunta-teste	Pedido pequeno que deveria acionar uma regra. Mostra se ela está sendo seguida.

8 Para ir *além*

- **code.claude.com/docs/en/memory**: ordem de carga, subpastas, `@imports`, comentários HTML, `AGENTS.md`, "My CLAUDE.md is too large" e o `/doctor prompt-audit`.
- **code.claude.com/docs/en/best-practices**: "Keep CLAUDE.md short" (com um exemplo enxuto em Node) e a tabela "What you can stop doing".
- **code.claude.com/docs/en/discover-plugins**: instalar plugin pelo terminal e pelo `/plugin`, e o marketplace oficial em máquina nova.
- **github.com/anthropics/claude-plugins-official**, pasta `plugins/claude-md-management`: o texto do `/revise-claude-md` e os critérios de nota do `claude-md-improver`.
- **arxiv.org/abs/2507.11538** (IFScale): o estudo das 500 instruções e do viés para as primeiras.

Próxima: *Aula 3.2 · Guidelines do seu projeto*

E quando uma ferramenta escreve o arquivo por você? O Laravel Boost escreve umas 160 linhas num comando. Na 3.2: o que o Boost põe lá dentro, o boostify para as outras stacks e a skill para construir o seu CLAUDE.md.